

CPBSD 方法深度整理 + 与我们 GCN Bundle Pricing 的关键差异

子项目： `revenue-management/CPBSD_GCN_acceleration_study`

目标：调研 **CPBSD (Component Pricing with a Bundle Size Discount)** 能否由我们的 GCN 方法加速。

0) 本次阅读范围与说明

- 文献 A（重点，严格参考原文）：
- `参考文献目录/Bundle Pricing/Component Pricing_Bundle Size Discount:Component Pricing with a Bundle Size Discount.pdf`
- 文献 B（对照）：
- `自己的Paper更新进展/Bundle Pricing_GCN:GCN-Based Bundle Pricing Strategies and Experiments.pdf`

说明： - 对 CPBSD 的 Problem Definition 与 Method Formulation 以下内容均基于原文 Section 3 与 Section 5（含公式编号）进行整理。 - 对我们 GCN 文档的对照，依据该文稿中“背景/策略设计/实验”部分的显式描述。

1) 两篇 Paper 的关键差异（你指定的三条主线）

1.1 问题背景差异：Mixed Bundling vs. Random Valuation

- **CPBSD（文献A）：**
- 背景是经典多产品捆绑定价（bundle pricing）框架下，面对 MB（Mixed Bundling）在 2^N 级别上的复杂度问题，提出结构化机制 CPBSD。
- 核心是：
 - 产品级价格 p_n （component pricing）
 - 加上仅依赖 bundle size 的折扣 d_s （bundle size discount）

- 需求侧建模是 **随机估值向量** $V=(V_1, \dots, V_N)$ ，来自联合分布 F ，客户在给定价格菜单下做最优购买。
- 我们 **GCN 文稿 (文献B)**：
 - 背景是把 Bundle Pricing 的组合爆炸问题交给“学习 + 优化”混合范式，尤其用 GCN 预测候选关系/概率，再做 MILP/LP/局部搜索加速。
 - 关注点更偏“求解加速与可扩展”：TPM/MPM/PPM/LPLS 等策略对可行空间剪枝与替代求解。
 - 仍然依赖随机估值/分群收益建模作为底层优化输入，但文章主贡献是算法工程与计算效率而非新机制定义。

一句话：- CPBSD 是“**机制层创新** (pricing form)”；- 我们 GCN 是“**求解层创新** (solver acceleration)”。

1.2 Setting/Assumption 差异：Nonadditive vs Additive

- **CPBSD 主体理论 (文献A)**：
 - 在主模型中采用 **additive buyer** 假设：bundle valuation 为所含产品估值之和。
 - 购买决策是 utility-maximization，且默认 tie-break in firm's favor。
- 原文还在扩展讨论中提到可引入 pairwise interaction 项 $[\sum_{n \in S} V_n + \sum_{\{i,j\} \in S, i \neq j} \alpha_{ij}]$ 其中 $\alpha_{ij} > 0$ 补性、 $\alpha_{ij} < 0$ 替代性；但这属于扩展场景，不是主干理论的基础假设。
- 我们 **GCN 文稿 (文献B)**：
 - 方法上更强调图结构对“产品/segment关系”的表达能力，可在学习层面对复杂关联（潜在 non-additive interaction）做近似编码。
 - 但文稿当前呈现的优化骨架，仍与传统 bundle pricing 的 segment-valuation 框架兼容，未给出像 CPBSD 那样完整可证的非加性理论体系。

结论：- CPBSD 主干是 **Additive+随机估值+机制约束**；- 我们 GCN 更像 **Data-driven approximation layer**，有潜力承接 non-additive 结构，但目前更多是计算策略而非完整行为理论闭环。

1.3 解决方法差异：Mechanism Optimization vs Learning-to-Optimize

- **CPBSD (文献A):**
- 先定义机制 (p_n, d_s)
- 再写出 firm-level expected profit optimization (CPBSD)
- 用 sample approximation 把期望问题离散化
- 推导 customer bundle-content selection 的 LP / Dual
- 构造单层 MILP (CPBSD-MILP) 并给出约束组(9)~(23)
- 外加大规模近似算法 (附录)
- **我们 GCN (文献B):**
- 用 GCN 预测产品-分群相关性/入选概率
- 通过阈值/Top-k/百分比剪枝构造候选 bundle space
- 结合 MILP 或 LP+local search 求价格
- 以“收入-时间”权衡评估方案

本质上： - CPBSD：先有机制，再做优化； - 我们 GCN：先做学习降维，再做优化。

2) CPBSD 的 Problem Definition (严格对应原文)

对应原文 Section 3 (The Model) 与 3.2 (Formulation)

2.1 基本集合与决策

- 产品集合：($\mathcal{N}=\{1,\dots,N\}$)
- 成本：(c_n)
- 客户可购买任意子集 ($S\subseteq \mathcal{N}$)，每个产品最多 1 件
- CPBSD 价格菜单：
- 产品级价格 (p_n)
- 按 bundle size 的折扣 (d_s)，其中 ($s=|S|$)

客户购买 bundle (S) 时支付：[$\text{Pay}(S)=\sum_{n\in S} p_n - s d_s$]

企业对 bundle (S) 的利润：[$\Pi(S)=\sum_{n\in S}(p_n-c_n)-s d_s$.]

2.2 需求与行为假设

- 客户估值向量 ($V=(V_1, \dots, V_N) \sim F$), F 为联合分布
- 主模型为 additive valuation
- 客户在给定 $((p, d))$ 下选择最大化总剩余 (valuation-payment) 的 bundle

原文给出的选择问题 (记 $(x_n(V) \in \{0, 1\})$) 可写作: $[x(V) = \arg \max_{\{\tilde{x}\}} \sum_{n \in \mathcal{N}} (V_n - p_n + d_n) \tilde{x}_n]$ subject to $[\sum_{n \in \mathcal{N}} \tilde{x}_n = s, \quad \tilde{x}_n \in \{0, 1\}.]$

2.3 Firm 的期望利润问题 (CPBSD)

原文目标 (等式(2)附近) 可概括为: $[\max_{p, d}; \mathbb{E}_{V \sim F} [\sum_{n \in \mathcal{N}} (p_n - c_n - d_n x_n(V)) x_n(V)]]$ 并满足 practical constraints 集合 ($\mathcal{P}$)。

关键 practical constraint (原文强调) 包括: - 非负: $(p_n \geq 0, d_n \geq 0)$ - 折扣随 bundle size 的一致性/次可加性限制 (避免拆单套利): $[s \geq s_1 + s_2 \Rightarrow d_s \geq d_{s_1} + d_{s_2}, \quad s_1 + s_2 = s.]$

3) CPBSD 的 Method Formulation (严格对应原文 MILP)

对应原文 Section 5.1 (MILP Formulation)

3.1 样本化问题 (CPBSD-sample)

为避免连续分布期望导致的无限约束, 原文使用 K 个客户样本估值 ($v^k = (v_1^k, \dots, v_N^k)$), 转为样本平均利润最大化: $[\max_{p, d} \frac{1}{K} \sum_{k \in \mathcal{K}} \sum_{n \in \mathcal{N}} (p_n - c_n - d_n x_n(v^k)) x_n(v^k),]$ $[\text{s.t. } (p, d) \in \mathcal{P}.]$

3.2 客户子问题 BCS / BCS-LP / BCS-dual

定义 $(x_{kns} \in \{0, 1\})$: 客户 k 在购买 size s bundle 时, 是否选择产品 n 。

BCS (bundle content selection): $[\max_{x_{kns}} \sum_n (v_n^k - p_n + d_n) x_{kns}]$ $[\text{s.t. } \sum_n x_{kns} = s, \quad x_{kns} \in \{0, 1\}.]$

其 LP relax (BCS-LP) 把二元改为 $(0 \leq x_{kns} \leq 1)$ 。

其对偶 (BCS-dual):
$$\begin{aligned} & [\min_{\alpha, \beta} \sum_n \beta_n] \\ & \text{s.t. } \alpha_k + \beta_n \geq v_n^k - p_n + d_s, \forall \alpha_k \text{ free}, \beta_n \geq 0. \end{aligned}$$

原文 Proposition 3: 三者最优值相同 (BCS = BCS-LP = BCS-dual)。

3.3 单层 MILP 的核心思想

为避免双层 + 双线性, 原文引入: - $q_{kns} = (p_n - d_s)x_{kns}$: 客户 k 在 size s 选择产品 n 的支付变量 - $y_{ks} \in \{0, 1\}$: 客户 k 是否选择 size s - (w_{ks}, w_k) : 客户 surplus 相关变量

并用 big-M 线性化: - $q_{kns} \geq p_n - d_s - M(1 - x_{kns})$ - $q_{kns} \leq p_n - d_s$

再配合: - 每位客户最多选一个 bundle size - bundle size 与 x_{kns} 一致 - 顾客最优性由 (BCS-dual) 上界约束与 surplus 约束共同保证 - practical constraints (折扣结构、非负、二元)

原文完整 MILP 约束编号为 (9)~(23), 并在 Theorem 4 证明该 MILP 对应样本问题最优解。

3.4 复杂度与可解性 (原文)

原文给出数量级: - CPBSD-MILP 变量/约束约为 $(O(KN^2))$ 与 $(O(KN^2 + K^2))$ - 相比 MB (指数级 bundle 枚举) 显著更可解; - 比 BSP 稍重, 但在规模上可接受; 并给出数值实验可解时间表现。

4) 对“能否用 GCN 加速 CPBSD”的直接研判

可以, 而且切入点明确:

1. 加速点 A: 候选 bundle content 降维
2. CPBSD 难点之一是 x_{kns} 的组合搜索。
3. 可用 GCN 先预测“客户 k -产品 n ”相关分数, 构造每个 k, s 的候选产品池, 减少 MILP 变量数。
4. 加速点 B: warm-start dual/size decisions
5. 对 y_{ks} (size选择) 和部分 x_{kns} 做 warm start, 可显著减少 B&B 节点。
6. 加速点 C: 近似算法的学习化替代

7. 原文附录已有 approximation algorithm；可把其规则替换为学习策略（policy network/GCN scorer），再用 MILP 修复可行性。

但要注意一致性： - 不能破坏 CPBSD 的机制结构（ p_n, d_s ）和关键 practical constraints； - 学习模块应作为“候选生成/剪枝/初值”层，而不是直接绕过机制最优性约束。

5) 建议下一步（可直接执行）

- 在本子项目新增：
- `data_schema.md`：定义 CPBSD 训练样本（ $v^k, c_n, p_n, d_s, x_{\{kns\}}, y_{\{ks\}}, q_{\{kns\}}$ ）
- `gcn_acceleration_design.md`：给出 A/B/C 三条加速路线的变量映射
- `milp_reduced_form.md`：把“GCN候选集约束”写成可直接跑的 reduced MILP

如果你同意，我下一步直接把这 3 个文件也建好，并给你第一版可跑伪代码。